

Challenge Question

Consider the following algorithm for determining if an array contains unique elements only.

```
function ONLYUNIQUE(A)  
  for i = 1 to A.size do  
    for j = i + 1 to A.size do  
      if A[i] = A[j] then return false  
  
  return true
```

1. What is the “big-O” complexity of ONLYUNIQUE?
2. Suppose you have access to a fast sorting algorithm that runs in $O(n \log n)$. Could you then devise a faster algorithm that solves the same problem as ONLYUNIQUE?

1. Similar to selection sort, the inner loop is performing different numbers of iterations on each iteration of the outer loop. Everything else is constant time, so we must focus on how many times we perform the inner loop. Let n be the size of the array A . Then,

$$\begin{aligned}\text{work} &= (n-1) + (n-2) + \cdots + 2 + 1 + 0 \\ &= 0 + 1 + 2 + \cdots + (n-1) + (n-2) \\ &= \sum_{i=0}^n i \\ &= \frac{(n-1)n}{2} \\ &\in O(n^2)\end{aligned}$$

2. Yes, assuming it is OK for us to modify the array A (or we have passed A by value, so we only modify a copy). First, sort A using the $O(n \log n)$ algorithm. In a sorted array, any duplicate values must be neighbors, so it suffices to loop over the array checking adjacent values for duplicates. Here’s the complete algorithm in pseudocode:

```
function ONLYUNIQUE2(A)  
  SORT(A)  
  for i = 1 to A.size - 1 do  
    if A[i] = A[i + 1] then  
      return false  
  return true
```

The loop runs $n - 1$ times, so the complexity overall is $O(n \log n + n - 1) = O(n \log n)$.